N9 Dashcam Server

Technical Documentation & Setup Guide

Device	N9 ADAS/DMS Dashcam
Device ID	088556000002
Protocol	JT/T 808-2019 + JT/T 1078-2016
Output	HLS (HTTP Live Streaming)
Stack	Python 3.13 + FFmpeg
Date	30 April 2026

1. Project Overview

This project builds a self-hosted server that receives live video from an N9 dashcam over 4G LTE, without relying on any third-party CMS platform. The camera speaks two Chinese transport ministry protocols — JT/T 808-2019 for device management and JT/T 1078-2016 for video streaming — and the server translates these into HLS (HTTP Live Streaming) that any browser or media player can consume.

1.1 Why We Built This

The N9 dashcam ships pre-configured to connect to vendor-hosted CMS servers (CMSV6/CMSV7). These servers are often unreliable, region-blocked, or require paid subscriptions. By running our own server we get:

- Full control over video data — no third party sees the feed
- No subscription fees
- Custom processing — AI, alerts, storage, whatever you want
- Direct integration with your own systems

1.2 System Architecture

The system has three logical layers:

Layer 1	JT808 TCP server (port 6608) — handles device registration, authentication, heartbeat, GPS
Layer 2	JT1078 TCP server (port 1078) — receives raw H.264 video packets from the camera
Layer 3	HTTP server (port 8080) — serves HLS playlists, video segments, and the web player

Data flow:

```
N9 Camera (4G LTE)
    |
    |-- TCP:6608 --> JT808 Server (registration + commands)
    |                    |
    |                    |-- sends 0x9101 command back to camera
    |                    |   telling it to push video to port 1078
    |
    |-- TCP:1078 --> JT1078 Server (raw RTP-like video packets)
                         |
                         |-- FrameAssembler (re-joins fragments)
                         |-- FFmpeg subprocess (stdin pipe)
                         v
                   HLS segments (.ts files)
```

```
                              v
                    HTTP:8080 --> Browser / VLC / Any player
```

2. Protocol Reference

2.1 JT/T 808-2019 (Signalling)

JT808 is a Chinese national standard for vehicle-mounted terminal communication. It handles everything except the video stream itself — registration, authentication, GPS location reporting, heartbeat keepalive, and control commands.

Frame Format

Every JT808 message is wrapped in a frame:

```
0x7E | Header (12 bytes) | Body (variable) | Checksum (1 byte) | 0x7E

Header breakdown:
  Bytes 0-1:   Message ID (uint16 big-endian)
  Bytes 2-3:   Body properties (uint16) - lower 10 bits = body length
  Bytes 4-9:   Phone/Device number (6 bytes BCD)
  Bytes 10-11: Serial number (uint16)
```

Byte Stuffing (Escaping)

Because 0x7E is used as a frame marker, any occurrence of it inside the frame payload must be escaped:

```
0x7E inside payload  ->  0x7D 0x02
0x7D inside payload  ->  0x7D 0x01
```

Message IDs Used

ID (hex)	Direction	Purpose
0x0100	Device -> Server	Registration
0x0102	Device -> Server	Authentication
0x0002	Device -> Server	Heartbeat (every 30s)
0x0200	Device -> Server	Location report (GPS)
0x1003	Device -> Server	Media channel info
0x8100	Server -> Device	Registration response
0x8001	Server -> Device	General ACK
0x9101	Server -> Device	Request video stream
0x9102	Server -> Device	Close video stream

2.2 JT/T 1078-2016 (Video Stream)

JT1078 defines how video and audio are streamed from the device to the server over a separate TCP connection. The device opens this connection after receiving a 0x9101 command from the server. Packets use an RTP-like format with a custom 36-byte header.

Packet Header (36 bytes)

```
Offset  Size  Field
0       4     Magic: 0x30 0x31 0x63 0x64
4       1     V(4bits) | P(1) | X(1) | CC(4bits)
5       1     M(1bit) | Payload Type (7bits)  -- codec ID
6       2     Sequence number (uint16 big-endian)
8       4     RTP timestamp (uint32, 90kHz clock)
12      4     SSRC (uint32)
16      6     SIM/Device number (BCD)
22      1     Channel number
23      1     Data type (upper 4) | Packet type (lower 4)
24      8     Unix timestamp milliseconds (uint64)
32      2     Last I-frame distance
34      2     Payload length (uint16)
36      N     Payload (H.264 NAL units)
```

Data Types & Packet Types

Data type 0x0	Video frame
Data type 0x1	Audio frame
Packet type 0x0	Complete frame (no fragmentation)
Packet type 0x1	First fragment
Packet type 0x2	Last fragment
Packet type 0x3	Middle fragment

Codec Payload Types

PT 98	H.264 (most common on N9)
PT 99	H.265 / HEVC
PT 4	G.711A (audio)
PT 8	G.711U (audio)

3. Code Walkthrough

3.1 jt808.py — Protocol Parser

This file is the foundation. It handles all JT808 framing — reading frames from raw TCP bytes and building response frames to send back to the device.

Key Functions

unescape(data)	Reverses byte-stuffing on raw frame bytes before parsing
escape(data)	Applies byte-stuffing when building outgoing frames
checksum(data)	XOR of all bytes — used to verify frame integrity
parse_frame(raw)	Takes raw bytes including 0x7E markers, returns JT808Message
build_frame(...)	Builds a complete outgoing frame with header + body + checksum
build_register_resp()	Builds 0x8100 registration response with auth code
build_general_resp()	Builds 0x8001 ACK for most device messages
build_av_request()	Builds 0x9101 command to tell device to start streaming video
parse_location(body)	Parses GPS coordinates, speed, heading from 0x0200 body

How parse_frame Works

```
1. Check raw[0] and raw[-1] are 0x7E (frame markers)
2. Call unescape() on everything between the markers
3. Verify XOR checksum of all bytes except the last
4. Parse fixed header fields (msg_id, body_props, phone, serial)
5. Check body_props bit 13 for sub-packet flag
6. Return JT808Message(header, body)
```

3.2 jt1078.py — Video Stream Parser

Handles the JT1078 video connection. Three classes work together to convert a raw TCP byte stream into complete H.264 frames ready for FFmpeg.

StreamBuffer

The fundamental problem with TCP is that data arrives in arbitrary chunks. StreamBuffer solves this:

```
1. Accumulate raw bytes in an internal bytearray
```

```
2. Search for the 4-byte magic (0x30316364)
3. Peek at offset 34 to read payload_length
4. Wait until we have 36 + payload_length bytes
5. Extract exactly that many bytes, advance buffer
6. Yield parsed JT1078Packet
```

FrameAssembler

Large video frames are split into fragments by the camera. FrameAssembler tracks fragment state and yields complete frames:

```
PT_RAW (0x0)    -> yield payload immediately (no fragmentation)
PT_FIRST (0x1)  -> start new buffer, store payload
PT_MIDDLE (0x3) -> append payload to buffer
PT_LAST (0x2)   -> append payload, yield complete frame, clear buffer
```

is_keyframe Detection

Keyframes (I-frames) are important for seeking and for FFmpeg to start encoding. Detection checks the H.264 NAL unit type:

```
Skip optional 0x00 0x00 0x00 0x01 Annex-B prefix
NAL type = first_byte & 0x1F
NAL type 5 = IDR slice (keyframe)
NAL type 7 = SPS (parameter set, also marks keyframe boundary)
```

3.3 hls.py — FFmpeg HLS Output

FFmpegStreamer wraps an FFmpeg subprocess per camera channel. It receives complete H.264 Annex-B frames and writes them to FFmpeg's stdin, which outputs HLS segments to disk.

FFmpeg Command

```
ffmpeg
  -loglevel warning
  -f h264          # input format: raw Annex-B H.264
  -i pipe:0        # read from stdin
  -c:v copy        # no re-encoding (passthrough)
  -f hls           # output format: HLS
  -hls_time 2      # 2-second segments
  -hls_list_size 5 # keep last 5 segments in playlist
  -hls_flags delete_segments+append_list
  -hls_segment_filename static/hls/<ch>/seg%05d.ts
  static/hls/<ch>/index.m3u8
```

> The -c:v copy flag is critical — it means FFmpeg does NOT re-encode the video. The camera already outputs H.264, so we just repackage it into .ts segments. This means zero CPU overhead for transcoding.

StreamManager

Manages one FFmpegStreamer per channel. Provides:

- get(channel) — returns or creates FFmpegStreamer for that channel
- write_frame(channel, frame) — async, pipes frame to correct FFmpeg process
- hls_url(channel) — returns the HTTP URL for the channel's playlist
- status() — returns dict of {channel: {live, frames}} for the /status API

3.4 main.py — Server Entry Point

Runs three asyncio TCP servers and one HTTP server concurrently. Uses Python's asyncio for all I/O — no threads, no blocking calls.

JT808Connection — Registration Flow

```
Device connects on port 6608
  |-- sends 0x0100 (Register)
  |       Server stores device, sends 0x8100 (OK, auth_code='ABCD1234')
  |-- sends 0x0102 (Auth with auth_code)
  |       Server sends 0x8001 (ACK)
  |       Server waits 0.5s
  |       Server sends 0x9101 (Request video on port 1078)
  |-- sends 0x0002 (Heartbeat every 30s)
  |       Server sends 0x8001 (ACK)
  |-- sends 0x0200 (GPS location)
          Server stores location, sends 0x8001 (ACK)
```

HTTP Server

A minimal hand-written HTTP/1.0 server (no frameworks) that serves:

GET /	`Web player page (HLS.js, 4-channel grid)`
GET /hls//index.m3u8	`HLS playlist for channel`
GET /hls//seg*.ts	`HLS video segments (MIME: video/mp2t)`
GET /status	`JSON: connected devices + stream health`

4. Setup Journey & Troubleshooting

4.1 What We Tried

Step 1 — Local WiFi Preview (UView)

The N9 creates a WiFi hotspot (SSID: N9-XXXXXX, password: 1122334455). Connecting your phone to this hotspot and opening UView gives a local preview over WiFi at 192.168.100.1. This worked immediately.

> The local WiFi preview does NOT use 4G or any server. It is a direct WiFi connection between your phone and the dashcam. This is only for configuration.

Step 2 — 4G Connectivity

The camera had an Airtel IoT SIM with APN airteliot.com. The fix was changing the APN to airtelgprs.com in UView Setup → 3G/4G. This gave the SIM open internet access instead of Airtel's private M2M network.

Step 3 — CMS Server Attempts

- 34.93.213.40:11010 — vendor default server, unreachable
- 120.24.66.218:6608 — public demo server, 404 error
- 8.222.192.111:6608 — briefly showed Linked, then dropped

> All third-party CMS servers are either dead, region-blocked, or require vendor credentials. This is why we built our own.

Step 5 — Public IP Solutions Attempted

ngrok TCP	`Requires credit card on free tier — blocked`
playit.gg TCP	`Requires premium — blocked`
Cloudflare Tunnel	`HTTP only, no raw TCP support`
Phone hotspot	`Phone was connected to N9 WiFi, couldn't also be a hotspot`

Step 6 — Recommended Next Step

The cleanest solution is a VPS with a public static IP:

- Oracle Cloud — always-free VM (e2.micro), requires card for verification only
- Google Cloud — $300 free credit for 90 days, e2-micro free after
- DigitalOcean / Linode — ~$6/month, cheapest paid option

> Once you have a public IP, point the camera's Server1 at it and the Python server will receive the connection immediately. The SIM is confirmed working.

5. Deployment Guide

5.1 Local (Windows) — Current Setup

```
# Python 3.13 from python.org, FFmpeg via: winget install ffmpeg
cd Desktop\n9server
python main.py --server-ip 192.168.x.x
# JT808:  0.0.0.0:6608
# JT1078: 0.0.0.0:1078
# HTTP:   http://192.168.x.x:8080
```

5.2 VPS / Cloud (Production)

```
sudo apt update && sudo apt install -y ffmpeg python3
scp -r n9server/ user@your-vps-ip:~/
cd ~/n9server && python3 main.py --server-ip <YOUR_VPS_PUBLIC_IP>
sudo ufw allow 6608/tcp   # JT808
sudo ufw allow 1078/tcp   # JT1078
sudo ufw allow 8080/tcp   # HTTP player
```

5.3 Camera Configuration

1. Setup -> 3G/4G -> APN: airtelgprs.com -> Save
2. Setup -> CMS -> Server1 IP:
3. Setup -> CMS -> Port1: 6608
4. Setup -> CMS -> Protocol1: JT808_19 -> Save
5. Check About tab — Server1 should show 'Linked' within 30 seconds

5.4 Viewing the Stream

Browser	`http://:8080/`
VLC (ch1)	`http://:8080/hls/1/index.m3u8`
VLC (ch2)	`http://:8080/hls/2/index.m3u8`
Status API	`http://:8080/status`

6. Troubleshooting Reference

Symptom	Fix
Server1: NoLi in About tab	Check APN is airtelgprs.com not airteliot.com. Check server IP is public.
4G: NoConnect	SIM not inserted or not activated. Try SIM in phone first.
All camera channels black	Tap the play button at top right of Video tab. Channels 3/4 need AHD cameras.
FFmpeg not found error	winget install ffmpeg. Restart Command Prompt after.
python not recognized	Add Python to PATH or use full path to python.exe.
JT808 connects but no video	FFmpeg may be missing. Check port 1078 is not firewalled.
Stream starts then freezes	Normal if no keyframe — recovers on next I-frame (~2s).
High latency (~10 seconds)	Reduce hls_time in hls.py from 2 to 1.
GPS: NoExist	Normal indoors. Camera needs sky visibility for GPS fix.
AiStatus: STR_Unauthorized	AI features need a vendor license. Doesn't affect video streaming.

7. Next Steps & Extensions

7.1 Immediate — Get Public IP

- Sign up for Oracle Cloud free tier (cloud.oracle.com)
- Create VM.Standard.E2.1.Micro in ap-mumbai-1 region
- Copy server files via SCP, install FFmpeg, run server
- Point camera at VPS IP — everything should work immediately

7.2 Future Enhancements

- Add RTSP output via FFmpeg for lower latency than HLS
- Store HLS segments to S3/GCS for cloud DVR / playback
- Parse GPS data and display on a map in the web player
- Add webhook alerts for ADAS events (FCW, LDW) from JT808 alarm messages
- Add audio support — JT1078 parser already handles audio packets
- Multi-device support — server already handles multiple devices by Device ID

> The server code is written to be extensible. Each camera channel gets its own FFmpeg process. Multiple cameras connect to the same JT808 port and are tracked by Device ID.

8. Quick Reference

Device Details

Device ID	`088556000002`
WiFi Hotspot	`N9-XXXXXX`
WiFi Password	`1122334455`
UView Password	`888888`
JT808 Port	`6608`
JT1078 Port	`1078`
HTTP Player Port	`8080`
APN	`airtelgprs.com`
Protocol	`JT808_19 + JT1078`

Server Files

main.py	Entry point — runs all 3 servers
jt808.py	JT808 protocol parser and frame builder
jt1078.py	JT1078 video packet parser and frame assembler
hls.py	FFmpeg HLS output manager